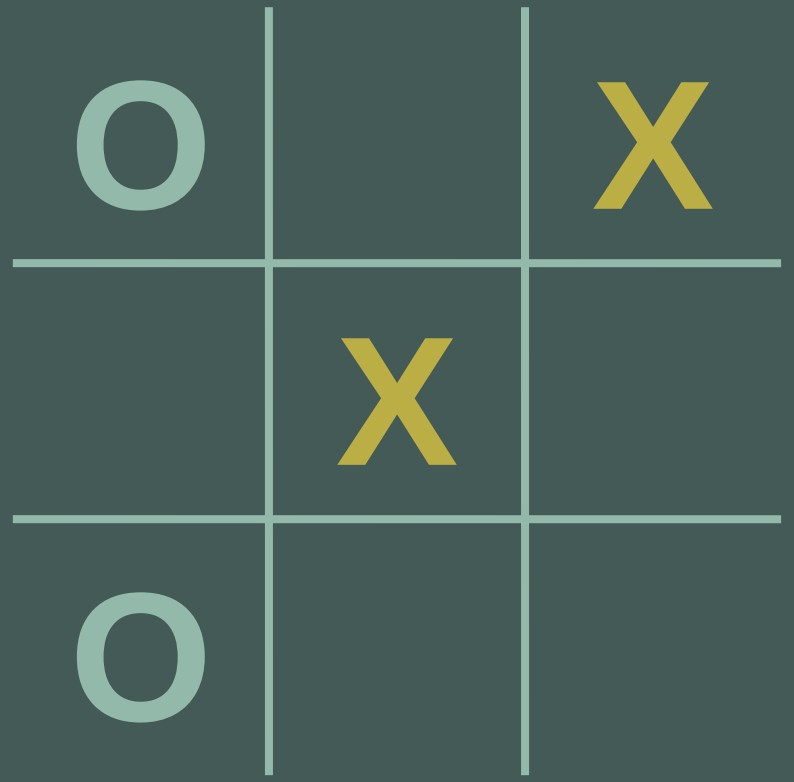
Adversarial search

Tic-tac-toe with minimax

The same game in Python and Prolog, side by side

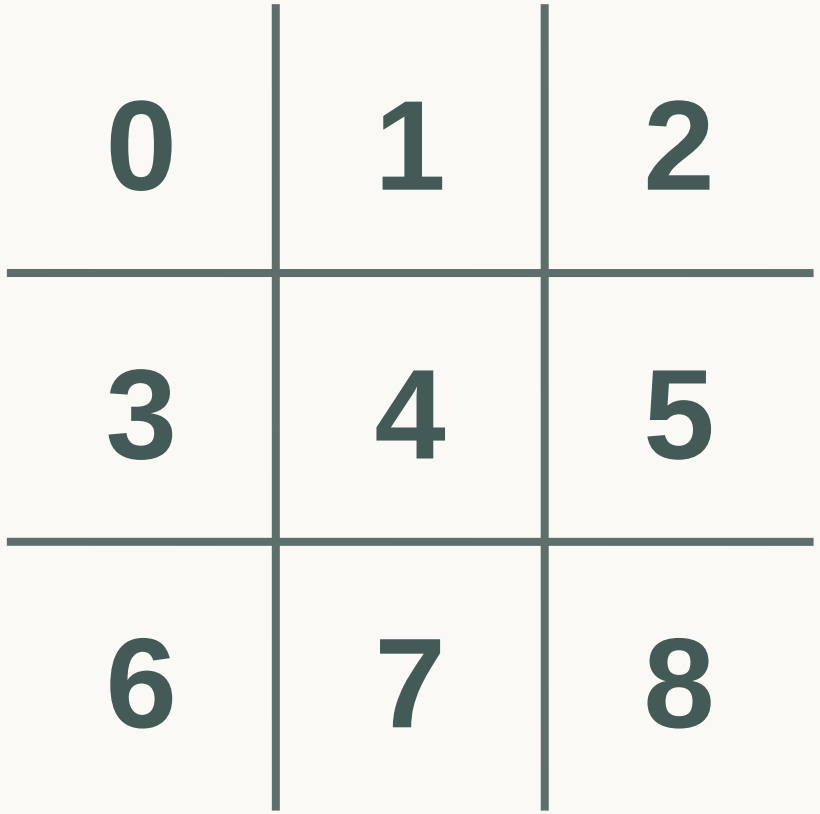


The game as a search problem

Slide 6	Python	Prolog
Initial state	`"-" * 9`	`[e,e,e,e,e,e,e,e,e]`
Successor function	`moves(), play()`	`moves/2, play/4`
Whose turn	`to_move()`	`to_move/2`
Terminal test	`terminal()`	`terminal/1`
Utility	`utility()  → 1, −1, 0`	`utility/2  → 1, −1, 0`

Each row of the definition becomes one function in Python and one predicate in Prolog, with the same name. minimax only talks to these five pieces, so it never needs to know it is playing tic-tac-toe.

Nine cells, numbered 0 to 8



cell = 3 × row + column

Python

```
board = "-" * 9          # empty
board = "o---x----"      # o in 0, x in 4
```

Prolog

```
Empty = [e,e,e,e,e,e,e,e,e],
Board = [o,e,e,e,x,e,e,e,e].
```

Both are read left to right, top to bottom. Python uses a string because it can be a dictionary key, which the memory trick will need; Prolog uses a list, with e for an empty cell.

Who has three in a row?

Python

```python
LINES = [(0, 1, 2), (3, 4, 5), (6, 7, 8),
         (0, 3, 6), (1, 4, 7), (2, 5, 8),
         (0, 4, 8), (2, 4, 6)]

def winner(board):
    for a, b, c in LINES:
        trio = board[a] + board[b] + board[c]
        if trio in ("xxx", "ooo"):
            return trio[0]
    return None
```

Prolog

```prolog
line(0,1,2). line(3,4,5). line(6,7,8).
line(0,3,6). line(1,4,7). line(2,5,8).
line(0,4,8). line(2,4,6).

winner(Board, P) :-
    line(A, B, C),
    nth0(A, Board, P), P \== e,
    nth0(B, Board, P), nth0(C, Board, P).
```

1. The eight winning lines: three rows, three columns, two diagonals.

2. Python loops over the lines; Prolog backtracks through line/3 until one has three equal marks.

Moves: where can I play, and what happens

Python

```
    def moves(board):
1       return [i for i in range(9)
                if board[i] == "-"]

    def play(board, i, player):
2       return board[:i] + player + board[i + 1:]
```

Prolog

```
    moves(Board, Moves) :-
1       findall(I, nth0(I, Board, e), Moves).

    play(Board, I, P, NewBoard) :-
2       nth0(I, Board, e, Rest),        % remove e
        nth0(I, NewBoard, P, Rest).     % insert P
```

1. The legal moves are the empty cells.

2. A move builds a new board; the old one never changes. In Prolog, nth0/4 removes the e and then inserts P.

Whose turn, is it over, who won

Python

```python
def to_move(board):
    x, o = board.count("x"), board.count("o")
    return "x" if x == o else "o"

def terminal(board):
    return (winner(board) is not None
            or "-" not in board)

def utility(board):
    scores = {"x": 1, "o": -1, None: 0}
    return scores[winner(board)]
```

Prolog

```prolog
to_move(Board, P) :-
    count(x, Board, NX), count(o, Board, NO),
    ( NX =:= NO -> P = x ; P = o ).

terminal(Board) :- winner(Board, _), !.
terminal(Board) :- \+ memberchk(e, Board).

utility(Board, U) :-
    (   winner(Board, x) -> U = 1
    ;   winner(Board, o) -> U = -1
    ;   U = 0
    ).
```

1. X moves when both players have the same number of marks.
2. The game is over if someone won or the board is full.
3. +1 if X wins, −1 if O wins, 0 for a draw.

minimax: the same four steps

Python

```
    def minimax(board):
        """Return (value, best_move)."""
1       if terminal(board):
            return utility(board), None
2       player = to_move(board)
3       pairs = []
        for i in moves(board):
            child = play(board, i, player)
            pairs.append((minimax(child)[0], i))
4       if player == "x":
            return max(pairs)
        return min(pairs)
```

Prolog

```
    minimax(Board, Val, Best) :-
1       (   terminal(Board)
        ->  utility(Board, Val), Best = none
2       ;   to_move(Board, P),
3           moves(Board, Moves),
            findall(V-I,
                    ( member(I, Moves),
                      play(Board, I, P, B2),
                      minimax(B2, V, _) ),
                    Pairs),
4           best(P, Pairs, Val-Best)
        ).

4   best(x, Pairs, Best) :-
        max_member(Best, Pairs).
    best(o, Pairs, Best) :-
        min_member(Best, Pairs).
```

① **leaf** ② **whose turn** ③ **value of every move** ④ **best pair.** max of (value, move) tuples breaks ties like max_member/2, so both agents choose the same move in all 447 positions we tested.

One line of memory

Python

```python
@lru_cache(maxsize=None)
def minimax(board):
```

Prolog

```prolog
:- table minimax/3.
minimax(Board, Val, Best) :-
```

549,946
→ 5,478

calls to minimax to solve the empty board

≈ 1 s
→ 0.02 s

Python, without and with the cache

≈ 5 s
→ 0.15 s

Prolog, without and with the table

Many move orders reach the same board. With memory, each board is solved only once: the transposition table of slide 30, for free.

Your turn, then the agent's

Python

```python
# inside the while loop of game()
player = to_move(board)
if player == "x":                    # you
    i = int(input("Move (0-8): "))
    if i not in moves(board):
        print("Not a legal move")
        continue
else:                                # the agent
    _, i = minimax(board)
    print(f"Agent plays {i}")
board = play(board, i, player)
```

Prolog

```prolog
choose(x, Board, I) :-               % you
    repeat,
      write('Move (0-8): '), read(I),
      moves(Board, Ms),
      (   memberchk(I, Ms) -> !
      ;   writeln('Not a legal move'), fail
      ).
choose(o, Board, I) :-               % the agent
    minimax(Board, _, I),
    format("Agent plays ~w~n", [I]).

% in turn/2, then recurse:
play(Board, I, P, Board2), turn(Board2).
```

1. You: read a cell and reject illegal moves.
2. The agent: ask minimax for its best move.
3. Play it and go on: a while loop in Python, recursion in Prolog.

Run it

Python

```
$ python3 gato.py
- - -
- - -
- - -
Move (0-8): 4
- - -
- x -
- - -
Agent plays 0
o - -
- x -
- - -
```

Prolog

```
$ swipl gato.pl
?- game.
- - -
- - -
- - -
Move (0-8): 4.
- - -
- x -
- - -
Agent plays 0
o - -
- x -
- - -
```

In Prolog, end every move with a dot: 4.

Play it in your browser



Choose who decides

The page runs the same gato.py and gato.pl inside the browser. Switch the agent between Python and Prolog in the middle of a game, show the minimax value of every free cell, or watch Python play against Prolog.

The first visit downloads the engines (about 12 MB for Python, 4 MB for Prolog).

ferdiex.github.io/gato

Try it 1: remove the memory

How much slower is the agent without memory? Predict first, then measure.

Python

```
# in gato.py, comment out one line:
# @lru_cache(maxsize=None)
def minimax(board):

# then, in a new console:
>>> import gato
>>> from timeit import timeit
>>> f = lambda: gato.minimax("-" * 9)
>>> timeit(f, number=1)
1.2571683490000005
```

Prolog

```
% in gato.pl, comment out one line:
% :- table minimax/3.

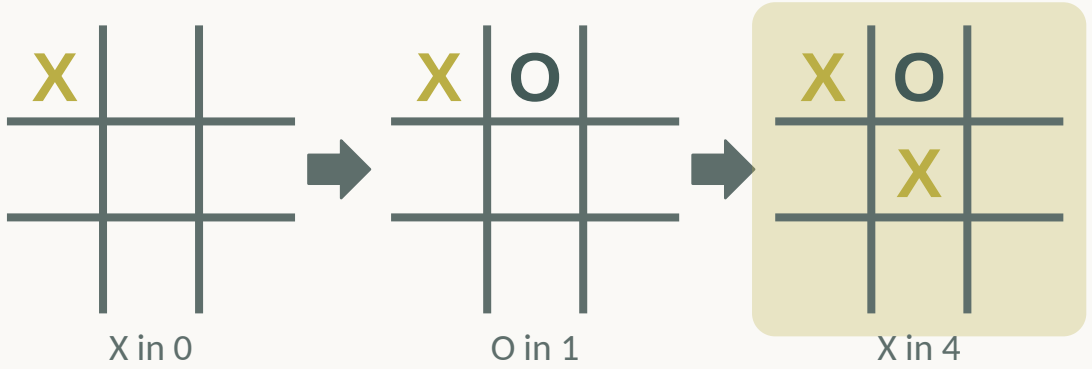
% then, at the ?- prompt:
?- make.
?- time(minimax([e,e,e,e,e,e,e,e,e], V, I)).
% 88,507,222 inferences, 4.843 CPU in 4.888 seconds
V = 0,
I = 8.
```

With the memory line: 0.017 s in Python; 1,527,662 inferences and 0.2 s in Prolog. Where does all the extra time go?

Try it 2: same board, different paths

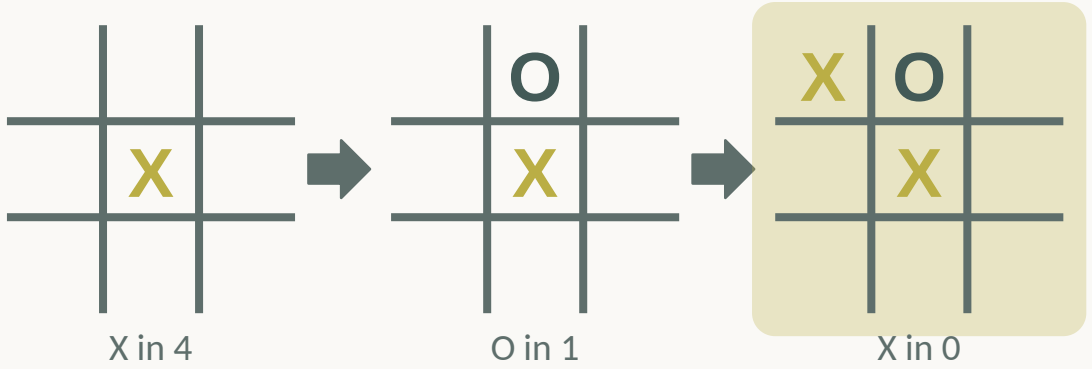
Without memory minimax makes 549,946 calls, but there are only 5,478 different boards. Why?

Path A



Both paths end in exactly the same board (shaded). Without memory, minimax solves it once for each path that reaches it, and everything below it too.

Path B



With memory, the second time it finds the board already solved. Find another pair of paths that meet.

Try it 3: let the agent start

If the agent plays X and moves first, can you ever beat it?

On the game page: Empieza → El agente. In the console, change who you are:

Python

```
# in game(), before:
if player == "x":                    # you

# after: now you play O
if player == "o":                    # you
```

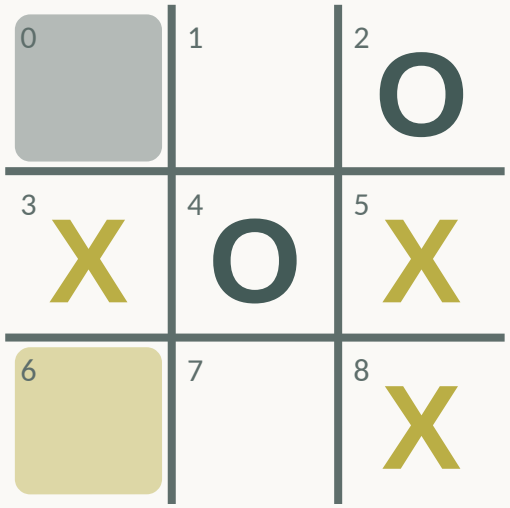
Prolog

```
% in gato.pl, before:
choose(x, Board, I) :-               % you
choose(o, Board, I) :-               % the agent

% after: swap x and o in the two heads
choose(o, Board, I) :-               % you
choose(x, Board, I) :-               % the agent
```

The agent opens with "Agent plays 8". Play as well as you can, and keep track of how each game ends.

Try it 4: make it win faster



O to move. Winning now is cell 6 (olive), but the agent picks cell 0 (grey): it also wins, just later.

With utilities of only +1 and −1, winning now or three moves later is worth the same, and the agent breaks the tie by the lowest cell number. This happens in 33 of the 447 positions where it decides. The fix: give earlier wins a larger value.

Python

```python
def utility(board):
    k = board.count("-") + 1
    scores = {"x": k, "o": -k, None: 0}
    return scores[winner(board)]
```

Prolog

```prolog
utility(Board, U) :-
    count(e, Board, Free), K is Free + 1,
    (   winner(Board, x) -> U = K
    ;   winner(Board, o) -> U is -K
    ;   U = 0
    ).
```

Same algorithm, two languages

Python walks the tree with loops; Prolog lets backtracking do it. Both agents play exactly the same moves.

0

The value of tic-tac-toe with perfect play: if nobody makes a mistake, it is a draw.

5,478

Different boards that can appear in a game. Memory solves each one only once.

0

Games lost by the agent, out of all the 101 possible games as X and 681 as O.